

Consenso Distribuito

Dal problema classico a Basic Paxos

Architetture dei Sistemi Distribuiti

Lezione 22

La lezione 21 ha separato ordine temporale e causalità.

Oggi studiamo un problema diverso:

come fanno più nodi a scegliere lo stesso valore?

Il caso di studio è Basic Paxos: una singola istanza di consenso che decide un solo valore.

Perché Non Basta Ordinare

Un clock logico può ordinare eventi.

Il consenso richiede una proprietà più forte:

tutti i nodi corretti che apprendono una decisione apprendono lo stesso valore

Esempio nel KV store:

$$\text{log}[42] = \text{SET } x \ 1$$

Se una replica apprende SET $x \ 1$ e un'altra SET $x \ 2$, la storia replicata è rotta.

Nel KV store distribuito il consenso può servire per:

- scegliere il leader corrente;
- decidere una entry del log replicato;
- decidere una configurazione valida del cluster;
- assegnare un lock distribuito;
- scegliere il valore corretto durante recovery.

Il punto non è produrre un timestamp. Il punto è impedire decisioni incompatibili.

```
propose(value) -> accepted_for_consensus  
learn()        -> chosen_value
```

Il contratto non promette che ogni proposta venga scelta.

Promette che, se un valore viene appreso, non sarà incompatibile con quello appreso da un altro learner corretto.

Un insieme di processi propone valori.

Il sistema deve scegliere un risultato.

Proprietà classiche:

- Validity;
- Agreement;
- Integrity;
- Termination.

Nel modello con guasti distingueremo sempre safety e liveness.

Solo valori proposti possono essere scelti e appresi.

$$\textit{learn}(l, v) \Rightarrow \textit{proposed}(v)$$

Esempio:

$$\textit{learn}(\log[42]=\text{SET } x \ 1)$$

deve derivare da una proposta reale.

Due learner non possono apprendere valori diversi.

$$\textit{learn}(l_1, v_1) \wedge \textit{learn}(l_2, v_2) \Rightarrow v_1 = v_2$$

Questa è la proprietà che protegge la storia replicata.

Se Agreement fallisce, il sistema può applicare comandi incompatibili.

Integrity

$$decide(p, v_1) \wedge decide(p, v_2) \Rightarrow v_1 = v_2$$

Termination

un valore viene prima o poi appreso, se restano abbastanza processi corretti

In sistemi asincroni con guasti, la termination richiede condizioni operative favorevoli.

Safety

- qualcosa di sbagliato non deve mai accadere;
- esempio: due valori diversi non vengono scelti.

Liveness

- qualcosa di buono deve prima o poi accadere;
- esempio: un valore viene scelto.

Paxos privilegia safety; il progresso dipende da condizioni pratiche.

Per Basic Paxos assumiamo processi che:

- procedono a velocità arbitraria;
- possono fallire;
- possono rientrare dopo crash se conservano stato stabile;
- non mentono;
- non colludono per sovvertire il protocollo.

Questi sono guasti non bizantini.

Crash-stop

- il nodo segue il protocollo finché funziona;
- poi smette di rispondere.

Crash-recovery

- il nodo può ripartire;
- deve conservare lo stato persistente necessario.

Bizantino

- il nodo può mentire o inviare messaggi contraddittori.

La rete:

- è asincrona;
- può ritardare i messaggi arbitrariamente;
- può perdere messaggi;
- può duplicare messaggi;
- può riordinare messaggi;
- non corrompe messaggi in modo malevolo.

Non esiste un timeout che distingua perfettamente lentezza e crash.

Con tre acceptor:

A1, A2, A3

i quorum di maggioranza sono:

$\{A1, A2\}$, $\{A1, A3\}$, $\{A2, A3\}$

Ogni quorum interseca ogni altro quorum.

L'intersezione trasferisce memoria tra tentativi diversi.

Con quorum di maggioranza, per tollerare F fallimenti simultanei si usa tipicamente:

$$N = 2F + 1$$

Esempi:

F	N	Quorum
1	3	2
2	5	3
3	7	4

- **Client**: richiede una decisione.
- **Proposer**: prova a far scegliere un valore.
- **Acceptor**: conserva promise e valori accettati.
- **Learner**: apprende il valore scelto.

Nelle implementazioni reali più ruoli possono stare nello stesso processo.

Nel ragionamento conviene tenerli separati.

Nel caso di un KV store:

```
client -> leader -> acceptor quorum -> learner -> replica
```

Il leader può agire da proposer.

Ogni slot del log replicato può essere una diversa istanza di consenso:

```
instance 42 decides log[42]
```

Basic Paxos decide un solo valore per una singola istanza.

Input:

- più valori possono essere proposti;
- più proposer possono competere;
- messaggi possono arrivare in ordini diversi;
- alcuni nodi possono fermarsi.

Output desiderato:

al massimo un valore viene scelto

Ogni tentativo usa un numero univoco e ordinabile:

$$n = (\text{round}, \text{proposer_id})$$

Il proposal number:

- non è il valore applicativo;
- non è un clock fisico;
- ordina i tentativi concorrenti;
- deve poter crescere dopo un round fallito.

Ogni acceptor conserva:

`promised_n`

`accepted_n, accepted_value`

- `promised_n`: massimo proposal number promesso;
- `accepted_n`: proposal number dell'ultimo valore accettato;
- `accepted_value`: valore associato a `accepted_n`.

Questo stato deve sopravvivere a crash e recovery.

Un round riuscito ha due fasi, ciascuna divisa in due passi:

Passo	Messaggio
Phase 1a	PREPARE(n)
Phase 1b	PROMISE(n, accepted_n, accepted_value)
Phase 2a	ACCEPT(n, value)
Phase 2b	ACCEPTED(n, value)

La fase 1 legge memoria. La fase 2 tenta di far accettare un valore sicuro.

Phase 1a: Prepare

Il proposer invia:

PREPARE(n)

ad almeno un quorum di acceptor.

Il valore applicativo non è necessario in questa fase.

Obiettivo:

- ottenere il diritto a proseguire;
- scoprire se il quorum conserva valori già accettati.

Phase 1b: Promise

Un acceptor riceve `PREPARE(n)`.

Se `n` è maggiore dei proposal number già promessi, risponde:

`PROMISE(n, accepted_n, accepted_value)`

La promise contiene:

- un vincolo sul futuro;
- memoria del passato.

Dopo $PROMISE(n, \dots)$ l'acceptor:

- non accetterà richieste con proposal number minore di n ;
- non concederà una nuova promise a PREPARE con numero non maggiore;
- riporta il valore già accettato con proposal number massimo, se esiste.

`accepted_n` e `accepted_value` impediscono di cancellare silenziosamente una decisione precedente.

Dopo promise da un quorum:

```
if no promise reports accepted_value:  
    value = own_value  
else:  
    value = value with highest accepted_n
```

Il proposer con proposal number più alto non sceglie sempre il proprio valore.

Sceglie il valore compatibile con la storia osservata nel quorum.

Phase 2a: Accept

Il proposer invia:

`ACCEPT(n, value)`

ad almeno un quorum di acceptor.

`n` è lo stesso proposal number della prepare riuscita.

`value` è il valore scelto con la regola del valore sicuro.

Phase 2b: Accepted

Un acceptor accetta `ACCEPT(n,value)` se non ha promesso di considerare solo numeri maggiori di `n`.

Se accetta:

`accepted_n = n`

`accepted_value = value`

Poi invia:

`ACCEPTED(n, value)`

Quando Un Valore È Scelto

Un learner apprende un valore quando osserva ACCEPTED da un quorum.

$$\text{chosen}(v) \Leftrightarrow \exists n, \exists Q : \forall a \in Q, \text{accepted}[a] = (n, v)$$

Il quorum è la soglia che rende la decisione stabile rispetto ai round successivi.

P1 -> A1,A2: PREPARE((1,P1))

A1 -> P1: PROMISE((1,P1), none, none)

A2 -> P1: PROMISE((1,P1), none, none)

P1 -> A1,A2: ACCEPT((1,P1), SET x 1)

A1 -> learners: ACCEPTED((1,P1), SET x 1)

A2 -> learners: ACCEPTED((1,P1), SET x 1)

{A1,A2} è un quorum, quindi SET x 1 è scelto.

A1: `accepted_n=(1,P1)`, `accepted_value=SET x 1`

A2: `accepted_n=(1,P1)`, `accepted_value=SET x 1`

A3: `accepted_n=none`, `accepted_value=none`

La decisione sopravvive anche se P1 cade.

La memoria è negli acceptor del quorum.

Arriva P2:

$n = (2, P2)$

valore desiderato = SET x 2

P2 invia:

P2 $\rightarrow$ A2, A3: PREPARE($(2, P2)$)

Il nuovo quorum $\{A2, A3\}$ interseca il quorum precedente $\{A1, A2\}$ in A2.

A2 -> P2: PROMISE((2,P2), (1,P1), SET x 1)

A3 -> P2: PROMISE((2,P2), none, none)

P2 vede che un valore era già stato accettato.

Quindi deve inviare:

ACCEPT((2,P2), SET x 1)

e non SET x 2.

- Ogni quorum successivo interseca un quorum precedente.
- Almeno un acceptor porta memoria della proposta già accettata.
- La promise espone `accepted_n` e `accepted_value`.
- Il proposer deve riproporre il valore con `accepted_n` massimo.

Il numero più alto consente di proseguire. Non consente di cambiare arbitrariamente il valore.

Un round può fallire se:

- non arrivano promise da un quorum;
- non arrivano accepted da un quorum;
- più proposer si superano continuamente;
- messaggi vengono persi o ritardati;
- acceptor sufficienti non sono raggiungibili.

Il proposer può riprovare con proposal number più alto.

Basic Paxos può non progredire se proposer concorrenti continuano a superarsi:

$$n=10, n=11, n=12, n=13, \dots$$

Per progresso pratico servono:

- un leader abbastanza stabile;
- quorum raggiungibile;
- retry con proposal number crescenti;
- rete che consegna messaggi per un intervallo sufficiente.

- `promised_n` di un acceptor non diminuisce.
- Un acceptor accetta (n, v) solo se non ha promesso un numero maggiore.
- Un proposer che vede valori accettati sceglie quello con `accepted_n` massimo.
- Un valore è scelto solo dopo `ACCEPTED` da un quorum.
- Due quorum di maggioranza si intersecano.

Questi invarianti sostengono Agreement.

Supponiamo che v sia scelto da un quorum Q .

Ogni round successivo che ottiene promise da un quorum Q' interseca Q .

Nell'intersezione trova memoria di v , o memoria di una proposta successiva che ha già incorporato v .

La regola del valore sicuro impedisce di scegliere un valore incompatibile.

Con $N = 3$, quorum = 2:

- un acceptor può non rispondere;
- il sistema può ancora ottenere un quorum;
- due acceptor non disponibili impediscono progresso;
- la safety resta protetta anche senza progresso.

La distinzione è importante:

non decidere è meglio che decidere due valori diversi

Paxos può decidere anche un valore come:

$$\text{leader} = P1$$

Se quel valore è scelto da un quorum, i learner corretti non apprendono un leader diverso per la stessa istanza.

In sistemi reali si usano varianti e ottimizzazioni, ma il cuore resta la protezione di Agreement.

Collegamento Con Log Replicato

Per un log replicato:

```
instance i -> log[i]
```

Ogni slot deve convergere su un solo comando.

Esempio:

```
instance 42 -> SET x 1
```

Multi-Paxos ottimizza l'esecuzione ripetuta di molte istanze.

Concetto	Funzione
Consenso	scegliere un valore comune
Agreement	impedire valori diversi
Quorum	trasferire memoria tra round
Promise	vincolare il futuro e riportare il passato
Accept	registrare un valore sicuro
Learner	apprendere quando vede un quorum

Clock e consenso risolvono problemi diversi.

- I clock aiutano a parlare di ordine e causalità.
- Paxos serve a far scegliere un valore condiviso.
- La safety dipende dagli invarianti, non dalla velocità della rete.
- La liveness dipende da condizioni operative sufficientemente stabili.

Domanda progettuale:

quale proprietà promette l'interfaccia distribuita?